

Team Project 1 - CAP

JavaScript Fundamental

Amjad



Chapter 1
Introduction to Javascript

Anjad

What is javascript?

- JavaScript (JS) is a high level, dynamic programming language widely used in web development.
- It is one of the three core web technologies:
  - HTML: Defines the structure and content of web pages.
  - CSS: Controls the appearance and styling.
  - JavaScript: Adds behavior, logic, and interactivity.
- JavaScript was created by Brendan Eich in 1995.
- JavaScript and Java are completely different programming languages.

Is JavaScript a Programming, Scripting, or Markup Language?

Type	Description	Example
Markup Language	Defines the structure of content	HTML
Programming Language	Provides instructions, logic, and algorithms	JavaScript, Java, Python
Scripting Language	A programming language commonly used to automate tasks and control software behavi or	JavaScript, Python

JavaScript is a programming language that is also commonly classified as a scripting language. It is **not** a markup language.



How Does JavaScript Execute?

Computers execute machine code, so programming languages need a way to translate or process their instructions.

Two common approaches are:

- **Interpreter**: Executes a program through a runtime environment, often processing instructions as the program runs.
- **Compiler**: Translates source code into another form, often machine code, before execution.

How JavaScript Works:

Modern JavaScript engines, such as Google's V8, use both interpretation and Just-In-Time (JIT) compilation to execute JavaScript efficiently.

- JavaScript is traditionally described as an interpreted language.
- Modern JavaScript engines also use compilation internally to improve performance.
- JavaScript runs inside web browsers and in other environments, such as Node.js.
- Interpreted and compiled are execution approaches, not mutually exclusive categories of programming languages.

Amjad

JavaScript in HTML

- JavaScript is the default scripting language in HTML.
- JavaScript was originally created to make web pages interactive and dynamic.
- JavaScript programs are called scripts.
- Scripts can be written directly inside HTML using the <script> element or loaded from external .js files.
- Scripts are executed by the browser's JavaScript engine.
- JavaScript does not require a separate manual compilation step to run in modern browsers.

```html
<!DOCTYPE html>
<html>
<head>
    <title>JavaScript Example</title>
</head>
<body>
    <h1 id="message">Hello!</h1>

    <script>
        document.getElementById("message").innerHTML = "Hello JavaScript!";
    </script>
</body>
</html>
```

Result: The heading changes from "Hello!" to "Hello JavaScript!" when the script executes.

What Can JavaScript Do?

JavaScript can dynamically control web pages and interact with users.

Capability	Description
Modify HTML Content	Add, remove, or update elements and text using the DOM
Change Attributes	Modify attributes such as src, href, and class
Change CSS Styles	Dynamically update colors, fonts, visibility, and layouts
Handle User Events	Respond to clicks, mouse movements, and keyboard input
Communicate with Servers	Send and receive data using Fetch API or AJAX techniques
Store Data	Save information using cookies and browser storage
Interact with Users	Display messages, alerts, and confirmation dialogs

```
// Change HTML content
document.getElementById("demo").innerHTML = "Welcome!";

// Change CSS style
document.getElementById("demo").style.color = "blue";

// Change an attribute
document.getElementById("photo").src = "newphoto.jpg";

// Display a message
alert("Hello JavaScript!");
```



What Makes JavaScript Unique?

- **HTML/CSS Integration**: JavaScript works directly with HTML elements and CSS styles.
- **Easy to Get Started**: Simple tasks can be implemented with a few lines of code.
- **Browser Support**: Supported by all major modern web browsers and generally enabled by default.
- **Immediate Execution**: JavaScript can run directly in the browser without a separate manual compilation process.
- **Event-Driven**: JavaScript can respond to user interactions and browser events.
- **Versatility**: JavaScript can be used for front-end, back-end, mobile, and desktop applications.

Amjad



Adding JavaScript to HTML

JavaScript can be integrated into HTML pages using two approaches:

1. **Internal JavaScript**: Write JavaScript code directly inside the HTML document using the <script> element.
2. **External JavaScript**: Write JavaScript code in a separate .js file and link it to the HTML document.

```
<!DOCTYPE html>
<html>
<body>
    <h2 id="demo">Hello!</h2>

    <script>
        document.getElementById("demo").innerHTML = "Hello JavaScript!";
    </script>
</body>
</html>
```

```
<h2 id="demo">Hello!</h2>
<script src="script.js"></script>
```

Where to Place JavaScript in HTML?

Option 1: Inside <head>

```
<!DOCTYPE html>
<html>
<head>
    <script src="script.js" defer></script>
</head>
<body>
    <h1>My Website</h1>
</body>
</html>
```

Using defer allows the browser to download th e script without blocking HTML parsing and exe cute it after the document has been parsed.

Option 2: At the End of <body>

```
<!DOCTYPE html>
<html>
<head>
    <title>My Website</title>
</head>
<body>
    <h1>My Website</h1>

    <script src="script.js"></script>
</body>
</html>
```

Placing the script at the end of <body> allows t he browser to process the page content before executing the script.

Inside <head> with defer	End of <body>
Script downloads while HTML is being parsed	Script is encountered after preceding HTML
Executes after HTML parsing finishes	Executes when encountered
Keeps script references organized	Simple and beginner-friendly
Recommended for many modern applications	Also a valid approach

Amjad

External JavaScript Files

External JavaScript is written in a separate file with the .js extension and linked to HTML using the src attribute of the <script> element.

```
<script src="script.js"></script>
```

Advantages of External JavaScript

- **Separation of concerns**: Keeps HTML structure separate from JavaScript logic.
- **Reusability**: The same JavaScript file can be used across multiple HTML pages.
- **Maintainability**: Makes code easier to read, edit, and debug.
- **Performance**: External scripts can be cached by browsers, reducing repeated downloads.

Amjad

JavaScript Comments

Comments are notes written inside source code that are ignored during JavaScript execution.

Why Use Comments?

- Explain the purpose and logic of code.
- Improve code readability and maintainability.
- Help other developers understand the code.
- Document important decisions or instructions.

Types of JavaScript Comments

1. Single-Line Comments (//)

Used to comment on a single line or part of a line.

```
// This is a single-line comment
let age = 25; // Store the user's age
```

2. Multi-Line Comments (/* ... */)

Used to write comments spanning multiple lines.

```
/*
This is a multi-line comment.
It can contain several lines.
*/
let name = "John";
```

Multi-line comments can also appear within a line:

```
let total = 10 /* first value */ + 20;
```

Amjad

JavaScript Output

JavaScript provides several ways to display information to users or developers.

Common JavaScript Output Methods:

Method	Description
alert()	Displays a popup message in the browser
console.log()	Prints information to the browser's developer console
document.write()	Writes content directly into the HTML document
innerHTML	Changes the HTML content inside an element

Examples

1. Using alert()

```
alert("Hello World!");
```

Displays a popup message with an OK button.

2. Using console.log()

```
console.log("Hello World!");
```

Displays a message in the browser's developer console.

3. Using document.write()

```
document.write("Hello World!");
```

Writes text into the HTML document.

4. Using innerHTML

```
document.getElementById("demo").innerHTML = "Hello World!";
```

Changes the content of an HTML element with id="demo".

Important Notes:

- alert() blocks interaction with the page until the user closes the dialog.
- console.log() is particularly useful for learning and debugging.
- document.write() is generally discouraged in modern web development, especially after a page has loaded, because it can replace the existing document.
- innerHTML should not be used with untrusted input without appropriate sanitization. Use textContent for plain text.

Amjad



Chapter 2
Variables & Data Types

Amjad



JavaScript Variables

What Is a Variable?

- A variable is a named container used to store data values in a program.
- Variables allow us to store, access, and manipulate data.
- A variable has a name and holds a value.
- Variables can store different types of data, such as numbers, strings, and booleans.
- JavaScript is dynamically typed, meaning variables are not restricted to a single data type.

```
let name = "John";
let age = 25;
let isStudent = true;

console.log(name);      // John
console.log(age);       // 25
console.log(isStudent); // true
```

Declaring Variables in JavaScript

JavaScript provides three keywords for declaring variables:

- `var` : The traditional way to declare variables (before ES6).
- `let` : Declares a variable whose value can be reassigned.
- `const` : Declares a variable whose binding cannot be reassigned.

Feature	var	let	const
Reassignment	Yes	Yes	No
Redeclaration in same scope	Yes	No	No
Scope	Function	Block	Block
Must initialize at declaration	No	No	Yes

```
// Using var
var city = "London";
city = "Paris"; // Allowed

// Using let
let age = 20;
age = 21; // Allowed

// Using const
const PI = 3.14159;
PI = 3.14; // TypeError
```

Best Practices

- Use const by default for values that do not need reassignment.
- Use let when the value needs to change.
- Avoid `var` in modern JavaScript code.

Anjad

JavaScript Variable Naming Rules

- Names can contain letters, digits, underscores (_), and dollar signs ($).
- Names cannot start with a digit.
- Names are case-sensitive (age and Age are different).
- Names cannot be reserved JavaScript keywords, such as let, class, or return.
- Spaces and hyphens are not allowed in variable names.

Naming Best Practices

- Use descriptive names: studentAge instead of x.
- Use camelCase for variables containing multiple words: firstName, totalPrice.
- Prefer meaningful English names for consistency.
- Avoid unnecessary abbreviations.

```
// Valid Names:
let firstName = "John";
let age2 = 25;
let _total = 100;
let $price = 50;
let studentAge = 20;

// Invalid Names:
let 2age = 25;
let first-name = "John";
let first name = "John";
let class = "A";
```

Anjad

var vs let

In JavaScript, both var and let are used to declare variables, but they have major differences in **scope**, **hoisting**, and **re-declaration**.

1. Scope (Where the variable is accessible)

- **var (Function-scoped):** If a var variable is declared inside a function, it is only accessible within that function. However, if it is declared inside a block (like an if statement or a for loop), it **leaks** out of that block and is accessible throughout the entire function or global scope.
- **let (Block-scoped):** A let variable is confined strictly to the block {} in which it is declared. This prevents variables from accidentally leaking out of loops or conditional statements.

```
function example() {
  if (true) {
    var x = "I am var";
    let y = "I am let";
  }
  console.log(x); // Works fine: "I am var"
  console.log(y); // ReferenceError: y is not defined
}
```

2. Hoisting (How the interpreter handles declarations)

- **var:** Variables declared with var are hoisted to the top of their enclosing scope and are automatically initialized with a value of undefined. This means you can access a var variable before it is declared in your code without getting an error (though its value will be undefined until the assignment line).
- **let:** Variables declared with let are also hoisted to the top of their block, but **they are not initialized**. Accessing a let variable before its declaration results in a ReferenceError. The time between entering the block and the actual declaration is known as the **Temporal Dead Zone (TDZ)**.

```
console.log(a); // Output: undefined (hoisted)
var a = 5;

console.log(b); // ReferenceError: Cannot access 'b' before initialization
let b = 10;
```

var vs let

In JavaScript, both var and let are used to declare variables, but they have major differences in **scope**, **hoisting**, and **re-declaration**.

3. Re-declaration in the same scope

- **var:** Allows you to re-declare the same variable multiple times within the same scope without any errors. This can easily lead to bugs where a variable is accidentally overwritten.
- **let:** Throws a SyntaxError if you try to re-declare the same variable within the same scope.

```
var count = 1;
var count = 2; // Perfectly legal with var

let name = "Alice";
let name = "Bob"; // SyntaxError: Identifier 'name' has already been declared
```

Summary Table

Feature	var	let
Scope	Function-scoped	Block-scoped
Hoisted?	Yes (initialized as undefined)	Yes (not initialized / Temporal Dead Zone)
Re-declaration	Allowed	Not allowed in the same scope
Global Object Property	Creates a property on the global window object	Does **not** create a property on the global object

Which one should you use?
In modern JavaScript (ES6 and later), **you should almost always use let** (or const if the variable won't be reassigned).
var is considered outdated and is generally only encountered in legacy codebases.

JavaScript Data Types

What Are Data Types?

A data type defines the kind of value stored in a variable and determines which operations can be performed on that value.

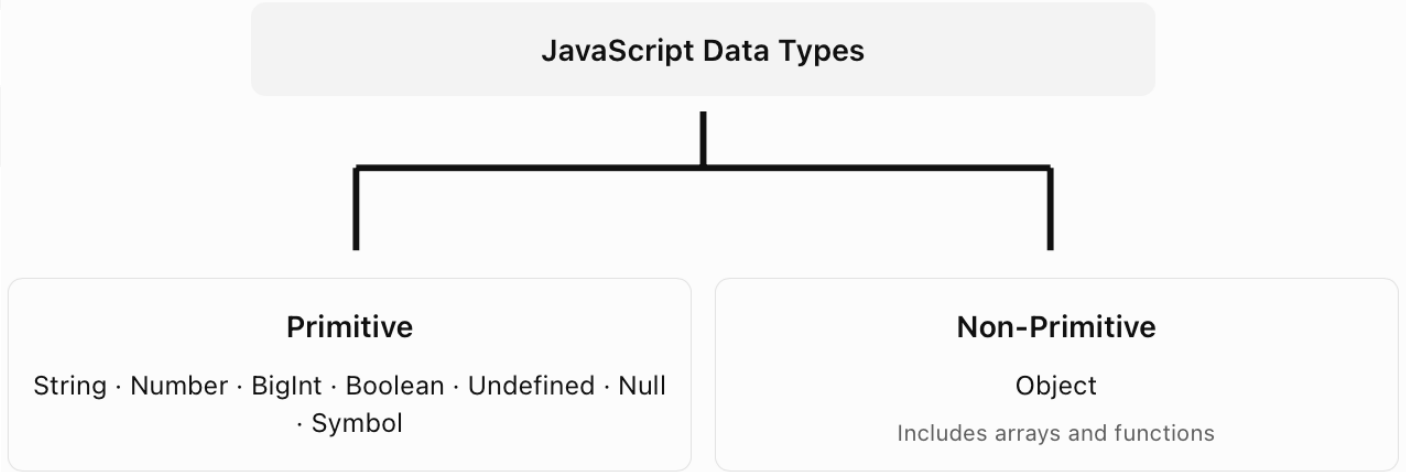
- JavaScript is a dynamically typed language.
- You do not need to specify a variable's data type when declaring it.
- JavaScript determines the type of a value at runtime.
- A variable can hold values of different types during program execution.

```
let name = "John";          // String
let age = 25;               // Number
let isStudent = true;       // Boolean
let address;                // Undefined
let salary = null;          // Null
```

JavaScript Data Types

JavaScript has 8 fundamental data types, divided into two categories:

Primitive Data Types	Non-Primitive Data Type
String	Object
Number	
BigInt	
Boolean	
Undefined	
Null	
Symbol	



JavaScript Primitive Data Types

Primitive data types represent basic values in JavaScript.

Data Type	Description	Example
String	Text enclosed in quotes	"Hello"
Number	Integers and decimal numbers	25, 3.14
BigInt	Integers of arbitrary precision	123456789n
Boolean	Logical values	true, false
Undefined	A value that has not been assigned	let x;
Null	Intentional absence of a value	let x = null;
Symbol	A unique identifier value	Symbol("id")

```
let studentName = "Sara";
let studentAge = 22;
let average = 89.5;
let graduated = false;
let phone;
let address = null;
```

Important Notes

- Strings can use single quotes ('Hello'), double quotes ("Hello"), or backticks (`Hello`).
- Both integers and decimals use the Number type.
- undefined and null are different:
- undefined: A value has not been assigned.
- null: A value has intentionally been set to represent no value.

Anjad

JavaScript Objects and the `typeof` Operator

Non-Primitive Data Types

- An object is a collection of properties and values. JavaScript objects can represent more complex data structures.
- Arrays and functions are also objects in JavaScript, although functions have a special `typeof` result.

```
// Object
let student = {
    name: "John",
    age: 25
};

// Array
let colors = ["Red", "Green", "Blue"];

// Function
function greet() {
    return "Hello!";
}
```

The typeof Operator

The `typeof` operator returns a string describing the type of a value.

```
console.log(typeof "Hello");   // "string"
console.log(typeof 25);        // "number"
console.log(typeof true);      // "boolean"
console.log(typeof undefined);// "undefined"
console.log(typeof {});        // "object"
console.log(typeof []);        // "object"
console.log(typeof greet);     // "function"

console.log(typeof null); // "object"
```



Chapter 3
Comparison & if statement

Amjad



Comparison Operators

What Are Comparison Operators?

Comparison operators are used to compare two values. The result of a comparison is always a Boolean value: true or false.

Operator	Description	Example	Result
==	Equal to (value, with type conversion)	5 == "5"	true
===	Strict equal to (no type conversion)	5 === "5"	false
!=	Not equal to (with type conversion)	5 != "5"	false
!==	Strict not equal to	5 !== "5"	true
>	Greater than	10 > 5	true
<	Less than	10 < 5	false
>=	Greater than or equal to	10 >= 10	true
<=	Less than or equal to	5 <= 3	false

```javascript
let age = 20;

console.log(age >= 18);    // true
console.log(age < 18);     // false
console.log(age == "20"); // true
console.log(age === "20");// false
```

Important: `==` vs. `===`

`==` Loose Equality

Allows type conversion

JavaScript

```javascript
5 == "5"
// true
```

Values match after conversion

`===` Strict Equality

Does not convert types

JavaScript

```javascript
5 === "5"
// false
```

Number ≠ String

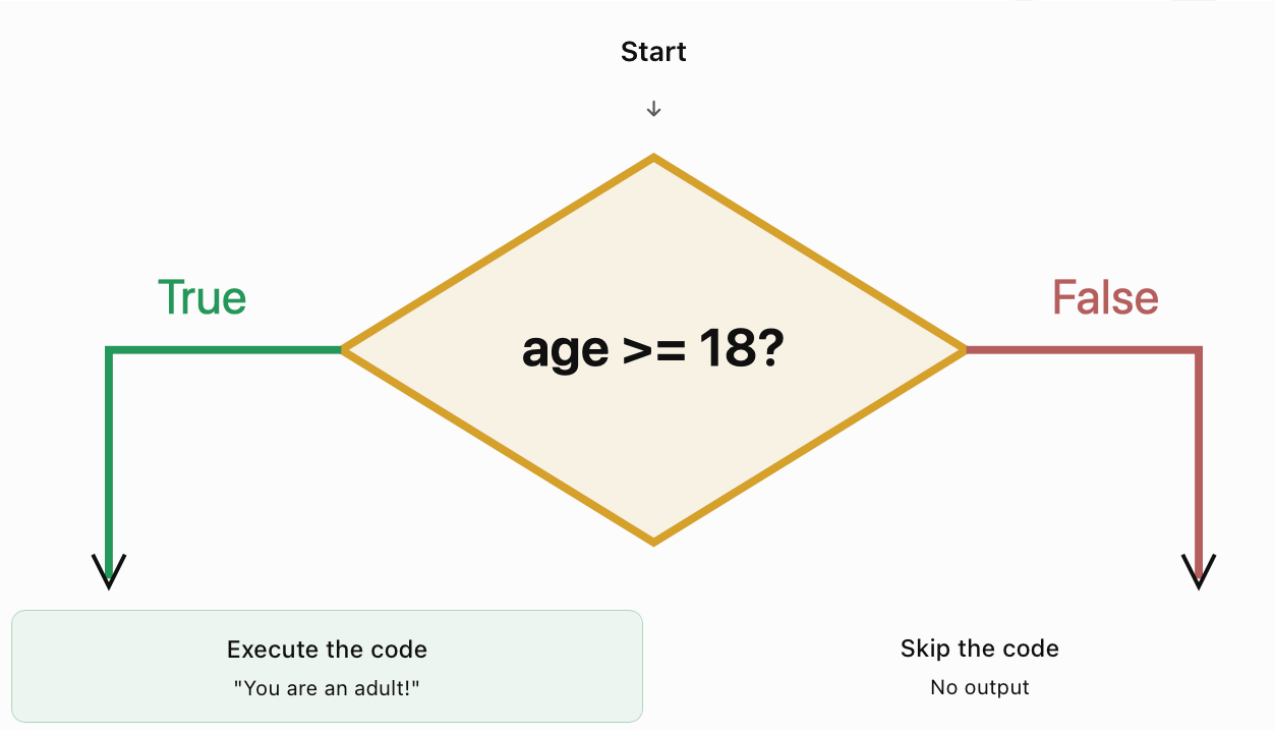
Best practice: Prefer `===` and `!==` over `==` and `!=` to avoid unexpected results caused by automatic type conversion.

The if Statement

What Is an if Statement?

The if statement is used to make decisions in JavaScript. It executes a block of code only if a specified condition is true.

- The condition is evaluated as true or false.
- If the condition is true, the code inside { } executes.
- If the condition is false, the code block is skipped.




```
let age = 20;

if (age >= 18) {
    console.log("You are an adult!");
} else {
    console.log("You are a child!");
}

// Result: You are an adult!
```



Chapter 4
Loops

Amjad



Javascript Loops

What Are Loops?

Loops are used to execute a block of code repeatedly as long as a specified condition is satisfied.

- Reduce code repetition.
- Automate repetitive tasks.
- Execute statements multiple times.
- Control repetition using conditions.

Types of JavaScript Loops

Loop	Description	Best Used When
for	Repeats code using initialization, condition, and update	Number of iterations is known
while	Repeats code while a condition is true	Number of iterations is unknown
do...while	Executes code once, then repeats while the condition is true	Code must execute at least once

Amjad

JavaScript for Loop

What Is a for Loop?

The for loop is used to repeat a block of code, typically when the number of iterations is **known**.

- Initialization: Runs once before the loop starts.
- Condition: Checked before each iteration. The loop continues while it is true.
- Update: Executes after each iteration.

```
for (initialization; condition; update) {
    // Code to execute
}
```

```
for (let i = 1; i <= 5; i++) {
    console.log(i);
}
```

How the loop executes

Iteration 1	Iteration 2	Iteration 3	Iteration 4	Iteration 5
1	2	3	4	5
i = 1	i = 2	i = 3	i = 4	i = 5

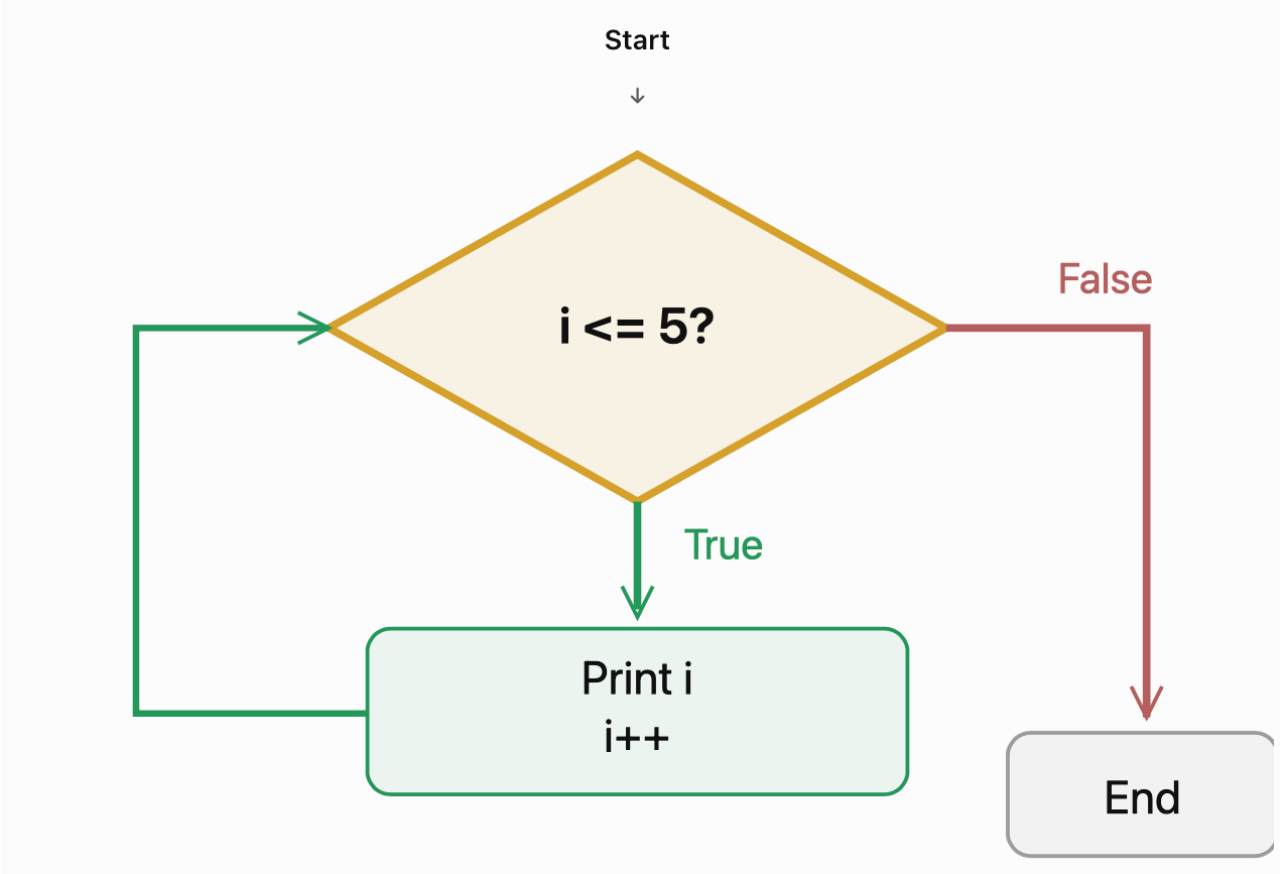
When `i = 6`, the condition `i <= 5` becomes false, and the loop stops.

JavaScript while Loop

What Is a while Loop?

The while loop executes a block of code repeatedly as long as a specified condition is true.

- The condition is checked before each iteration.
- If the condition is initially false, the loop never executes.
- Commonly used when the number of iterations is not known in advance.




```
while (condition) {
    // Code to execute
}

let i = 1;

while (i <= 5) {
    console.log(i);
    i++;
}
```

JavaScript do...while Loop

What Is a do...while Loop?

The do...while loop executes a block of code at least once, then repeats it as long as the condition remains true.

- The code block executes before checking the condition.
- The condition is evaluated after each iteration.
- Useful when an operation must run at least once.

```
do {
    // Code to execute
} while (condition);

let i = 1;

do {
    console.log(i);
    i++;
} while (i <= 5);
```

Key Difference: `while` vs. `do...while`

`while` Loop

JavaScript

```
let x = 10;

while (x < 5) {
    console.log(x);
    x++;
}
```

Output:

No output

The condition is false from the beginning.

`do...while` Loop

JavaScript

```
let x = 10;

do {
    console.log(x);
    x++;
} while (x < 5);
```

Output:

10

The code executes once before checking the condition.

Always ensure that the loop's condition can eventually become false. Otherwise, you may create an infinite loop.



Chapter 4
Functions

Amjad



Introduction to JavaScript Functions

What Is a Function?

A function is a reusable block of code designed to perform a specific task.

- Functions help organize code into smaller, manageable blocks.
- A function executes when it is called (invoked).
- Functions can receive input values called parameters.
- Functions can return a result using the return statement.
- Functions reduce code duplication and improve readability.

Defining a Function

A function must be defined before it can perform its task. A function declaration uses the function keyword, followed by a name, parentheses (), and a block of code {}
.

Key Concepts

- Function Declaration: Defines what the function does.
- Function Call: Executes the function.
- Reusability: A function can be called multiple times without rewriting its code.
- Hoisting: Function declarations can generally be called before their declaration appears in the code.

```
// Defining a Function
function functionName() {
    // Statements
}

functionName(); // Call the function

// Ex:
function sayHello() {
    console.log("Welcome to JavaScript!");
}

// Calling a Function Multiple Times
sayHello();
sayHello();
sayHello();
```

Anjal

Function Parameters and Arguments

What Are Parameters and Arguments?

Functions can accept input values to perform operations on different data.

- **Parameters**: Variables listed in the function definition.
- **Arguments**: Actual values passed to the function when calling it.
- A function can accept zero, one, or multiple parameters.

```
// Syntax
function functionName(parameter1, parameter2) {
    // Use the parameters
}

functionName(argument1, argument2);

// Example
function greet(name) {
    console.log("Hello, " + name + "!");
}

greet("Ali");
greet("Sara");

function add(a, b) {
    console.log(a + b);
}

add(10, 5); // 15
add(20, 8); // 28
```

The return Statement

What Is the `return` Statement?

The return statement is used to send a value back to the code that called the function.

- Returns a result from a function.
- Immediately stops the function's execution.
- Allows the returned value to be stored in a variable or used in another expression.
- A function without an explicit returned value returns undefined.

```
// Syntax
function functionName() {
    return value;
}

// Example
function add(a, b) {
    return a + b;
}

let result = add(10, 5);
console.log(result);
```

JavaScript Function Expressions

What Is a Function Expression?

A function expression is a function defined as part of an expression, commonly by assigning it to a variable.

- Functions can be stored in variables.
- Function expressions can be anonymous (without a function name).
- The function can be called using the variable name.
- Unlike function declarations, function expressions cannot generally be called before their initialization.

```
const greet = function(name) {
    return "Hello, " + name;
};

console.log(greet("Ali"));
```

Function Declaration vs. Function Expression

Function Declaration	Function Expression
Defined using function name()	Commonly assigned to a variable
Can be called before its declaration in the same scope	Must be initialized before calling
Creates a function binding	Creates a function value that can be assigned

Function Declaration

JavaScript

```
function add(a, b) {
    return a + b;
}
```

Function Expression

JavaScript

```
const add = function(a, b) {
    return a + b;
};
```

Both functions can be called using `add(5, 3)` and return `8`.

JavaScript Arrow Functions (ES6)

An arrow function is a shorter syntax for writing functions, introduced in ECMAScript 2015 (ES6).

- Uses the arrow operator `=>`.
- Provides a concise way to write functions.
- Can omit `{}` and `return` for simple single-expression functions.
- Arrow functions have different behavior from regular functions for `this` and cannot be used as constructors.

Regular Function

```javascript
const add = function(a, b) {
    return a + b;
};
```

Arrow Function

```javascript
const add = (a, b) => {
    return a + b;
};
```

Shorter Arrow Function Syntax

When the function contains only one expression, the braces and return can be omitted.

```javascript
const add = (a, b) => a + b;

console.log(add(10, 5)); // 15
```

For a single parameter, parentheses are optional:

```javascript
const square = x => x * x;

console.log(square(4)); // 16
```

1. Function Expression

```
const double = function(x) { return x * 2; };
```

↓

2. Arrow Function

```
const double = (x) => { return x * 2; };
```

↓

3. Concise Arrow Function

```
const double = x => x * 2;
```

JavaScript Variable Scope - Local vs. Global

What Is Variable Scope?

Scope determines where a variable can be accessed within a JavaScript program.

Types of Scope

- Global Scope: Variables declared outside functions and blocks are generally accessible throughout the script.
- Function Scope: Variables declared inside a function are accessible only within that function.
- Block Scope: Variables declared with let or const inside {} are accessible only within that block.

Scope Comparison

Scope	Declared Where?	Accessible Where?
Global	Outside functions and blocks	Throughout the relevant script environment
Function	Inside a function	Within that function
Block	Inside {} using let or const	Within that block





Chapter 5
Arrays & Objects

Amjad



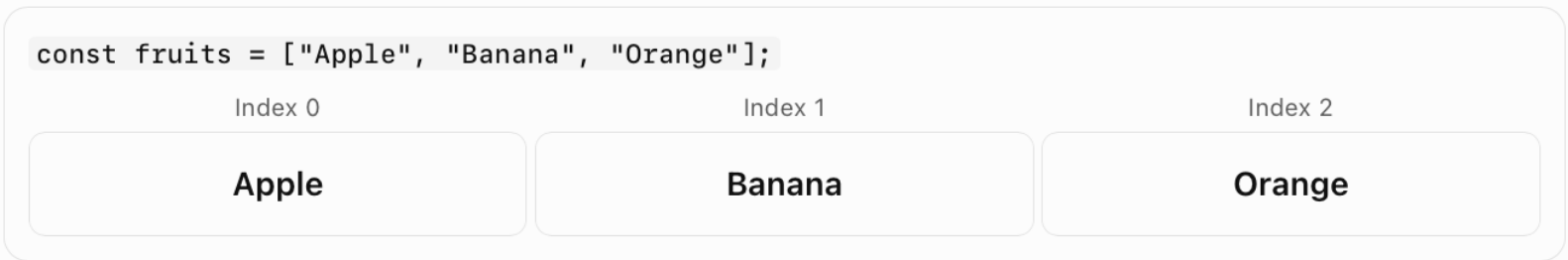
JavaScript Arrays

An array is a data structure used to store multiple values in a single variable.

- Arrays are created using square brackets [].
- Array elements are separated by commas.
- Arrays can store values of different data types.
- Each element has an index, starting from 0.
- Arrays can grow or shrink dynamically.

```
const fruits = ["Apple", "Banana", "Orange"];

console.log(fruits);
console.log(fruits[0]); // Apple
console.log(fruits[2]); // Orange
```



Accessing and Modifying Arrays

Accessing Array Elements: Use the element's index to access its value.

```
const colors = ["Red", "Green", "Blue"];

console.log(colors[0]); // Red
console.log(colors[1]); // Green
```

Modifying Array Elements: Array elements can be changed by assigning a new value to an index.

```
const colors = ["Red", "Green", "Blue"];

colors[1] = "Yellow";

console.log(colors); // ["Red", "Yellow", "Blue"]
```

Array Length: The length property returns the number of elements in an array.

```
const fruits = ["Apple", "Banana", "Orange"];

console.log(fruits.length); // 3
```

Common JavaScript Array Methods

JavaScript provides built-in methods to manipulate arrays.

Method	Description	Example
push()	Adds an element to the end	fruits.push("Mango")
pop()	Removes the last element	fruits.pop()
unshift()	Adds an element to the beginning	fruits.unshift("Kiwi")
shift()	Removes the first element	fruits.shift()
includes()	Checks whether an element exists	fruits.includes("Apple")
indexOf()	Returns the index of an element	fruits.indexOf("Banana")

```
let fruits = ["Apple", "Banana"];

fruits.push("Orange");
console.log(fruits);
// ["Apple", "Banana", "Orange"]

fruits.pop();
console.log(fruits);
// ["Apple", "Banana"]
```

Looping Through Arrays

Loops are commonly used to access and process array elements.

```
const fruits = ["Apple", "Banana", "Orange"];

for (let i = 0; i < fruits.length; i++) {
    console.log(fruits[i]);
}
```

Using a for...of Loop

```
const fruits = ["Apple", "Banana", "Orange"];

for (const fruit of fruits) {
    console.log(fruit);
}
```

Both examples produce the same output.

JavaScript Objects

An object is a collection of related data stored as key-value pairs, called properties.

- Objects are created using curly braces {}.
- Each property consists of a name (key) and a value.
- Properties are separated by commas.
- Objects can store different data types, including functions.

```
const student = {
    name: "Ali",
    age: 22,
    major: "Computer Science",
    isGraduated: false
};
```

Anjad

Accessing and Modifying Objects

JavaScript provides two common ways to access object properties

1. Dot Notation

```
console.log(student.name); // Ali
console.log(student.age);  // 22
```

2. Bracket Notation

```
console.log(student["name"]); // Ali
console.log(student["age"]);  // 22
```

Modifying and Adding Properties

```
const student = {
    name: "Ali",
    age: 22
};

// Modify an existing property
student.age = 23;

// Add a new property
student.city = "Nablus";

console.log(student);
```

Arrays vs. Objects

Arrays	Objects
Store ordered collections of values	Store data as key-value properties
Access elements using numeric indexes	Access properties using keys
Use square brackets []	Use curly braces {}
Example: fruits[0]	Example: student.name



Chapter 6
DOM

Amjad

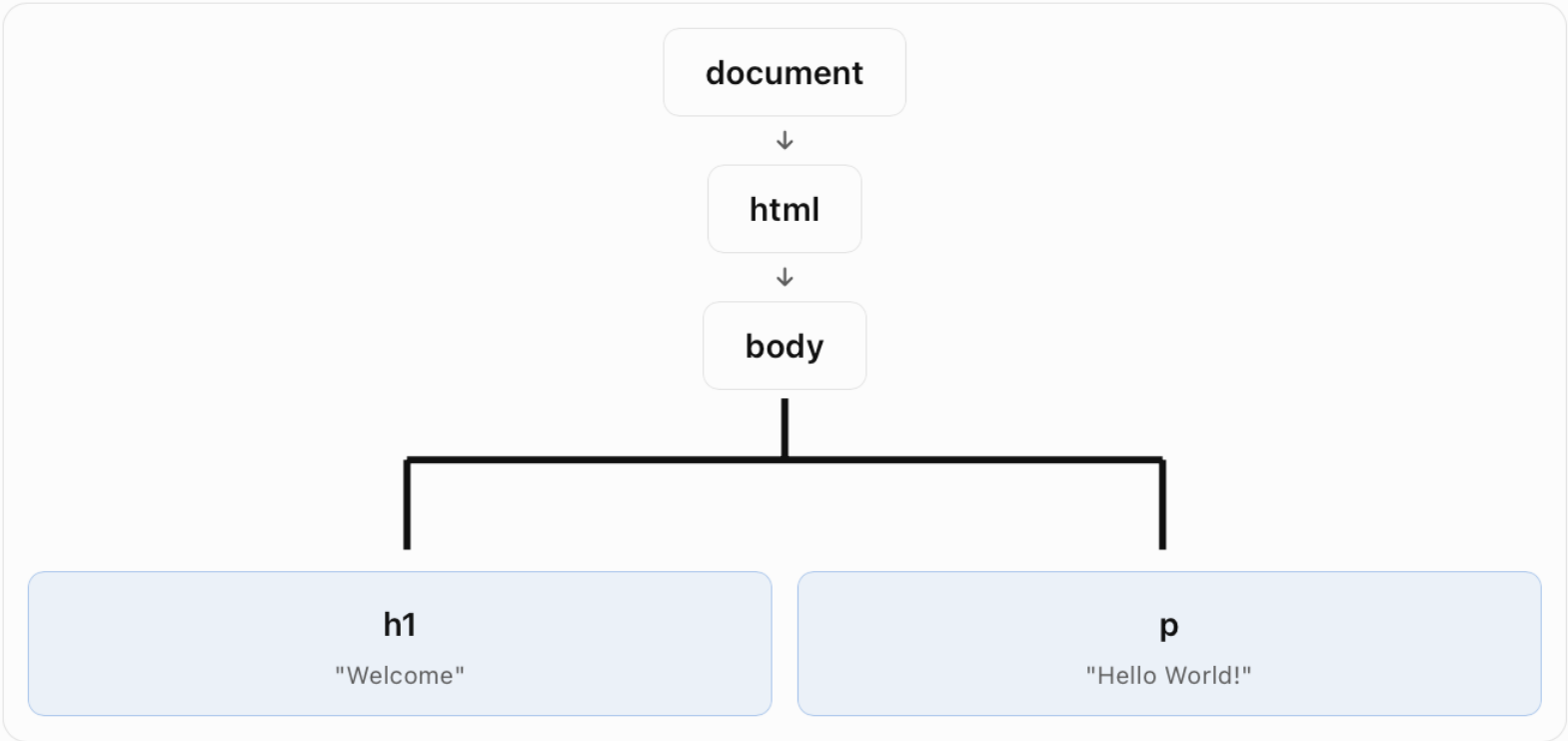


Introduction to the DOM

The **Document Object Model (DOM)** is a programming interface that represents an HTML document as a tree of objects.

- When a browser loads an HTML page, it creates a DOM representation of the document.
- Each HTML element becomes an object (node) in the DOM tree.
- JavaScript can access and manipulate these elements through the DOM.
- The DOM allows web pages to change dynamically without reloading.

```
<html>
<body>
    <h1>Welcome</h1>
    <p>Hello World!</p>
</body>
</html>
```



Selecting HTML Elements

Before modifying an HTML element, JavaScript must first select it from the DOM.

Common DOM Selection Methods

Method	Description
getElementById()	Selects an element by its ID
getElementsByClassName()	Selects elements with a specified class
getElementsByTagName()	Selects elements by their HTML tag name
querySelector()	Selects the first element matching a CSS selector
querySelectorAll()	Selects all elements matching a CSS selector

Important Notes

`.querySelector()` returns the first matching element, or null if none exists.
`.querySelectorAll()` returns a static NodeList of matching elements.
`.getElementsByClassName()` and getElementsByTagName() return live HTMLCollection

```html
<h1 id="title">Welcome</h1>
<p class="message">Hello!</p>
<p class="message">Good morning!</p>
```

```js
// Select by ID
const title = document.getElementById("title");

// Select by class
const messages = document.getElementsByClassName("message");

// Select using CSS selectors
const firstMessage = document.querySelector(".message");

// Select all matching elements
const allMessages = document.querySelectorAll(".message");
```

Amjad

Changing HTML Content

JavaScript can dynamically change the content of HTML elements.

Common Properties

Property	Description
innerHTML	Gets or sets HTML markup inside an element
textContent	Gets or sets the text content of an element
innerText	Gets or sets rendered text, taking styling and layout into account

```
<h2 id="demo">Original Text</h2>

<script>
// Change text content
document.getElementById("demo").textContent = "Hello JavaScript!";

// Change HTML content
document.getElementById("demo").innerHTML = "<em>Welcome!</em>";
</script>
```

Important: Use `textContent` when inserting plain text. Avoid inserting untrusted user input directly into `innerHTML`, as it can introduce security vulnerabilities.

Changing CSS Styles and Attributes

JavaScript can modify the appearance and attributes of HTML elements.

1. Changing CSS Styles: Use the .style property to modify inline CSS styles.

```
<p id="message">Hello World!</p>

<script>
const message = document.getElementById("message");

message.style.color = "blue";
message.style.fontSize = "24px";
message.style.backgroundColor = "yellow";
</script>
```

2. Changing HTML Attributes: JavaScript can read or modify HTML attributes.

```
<img id="photo" src="old.jpg" alt="Photo">

<script>
const photo = document.getElementById("photo");

photo.src = "new.jpg";
photo.alt = "Updated Photo";
</script>
```

3. Adding and Removing CSS Classes

```
message.classList.add("highlight");
message.classList.remove("highlight");
message.classList.toggle("active");
```

Creating and Removing HTML Elements

JavaScript can dynamically add or remove elements from a web page.

Creating a New Element

```
<div id="container"></div>

<script>
// Create a new paragraph
const paragraph = document.createElement("p");

// Add text to the paragraph
paragraph.textContent = "This is a new paragraph!";

// Add it to the page
document.getElementById("container").appendChild(paragraph);
</script>
```

Removing an Element

```
const paragraph = document.createElement("p");
paragraph.remove();
```

Other Useful Methods

Method	Description
createElement()	Creates a new HTML element
appendChild()	Adds a node as the last child
prepend()	Inserts content at the beginning of an element
remove()	Removes an element from the DOM
replaceWith()	Replaces an element with another node or text

Practical DOM Manipulation Example

Example: Change Text and Style Dynamically: This example combines selecting elements, modifying text, and changing CSS styles.

```html
<!DOCTYPE html>
<html>
<body>
    <h2 id="title">Welcome!</h2>
    <button onclick="changeContent()">Click Me</button>

    <script src="script.js"></script>
</body>
</html>
```

```javascript
function changeContent() {
    const title = document.getElementById("title");

    title.textContent = "Hello JavaScript!";
    title.style.color = "blue";
    title.style.fontSize = "30px";
}
```

What Happens?

1.JavaScript selects the heading using getElementById().
2.The heading text changes using textContent.
3.The heading color and font size change using .style.



Chapter 7
Javascript Events

Amjad



Introduction to JavaScript Events

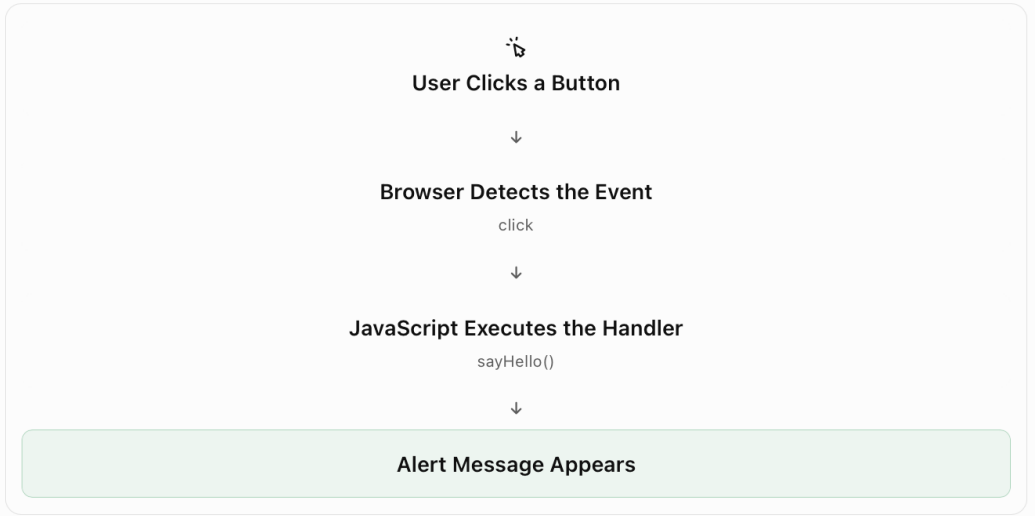
- An **event** is an action or occurrence detected by the browser, such as a user clicking a button, typing on a keyboard, or submitting a form.
- JavaScript can respond to these events by executing a function called an **event handler**.

Common JavaScript Events

Event	Description
click	Occurs when an element is clicked
dblclick	Occurs when an element is double-clicked
mouseover	Occurs when the mouse pointer enters an element or its descendants
mouseout	Occurs when the mouse pointer leaves an element or its descendants
keydown	Occurs when a keyboard key is pressed
input	Occurs when the value of an input changes through user interaction
change	Occurs when a value change is committed
submit	Occurs when a form is submitted
load	Occurs when a page or resource finishes loading

```html
<button onclick="sayHello()">Click Me</button>

<script>
    function sayHello() {
        alert("Hello JavaScript!");
    }
</script>
```



HTML Event Attributes

HTML elements can contain event attributes that execute JavaScript code when a particular event occurs.

Common attributes include:

- `onclick` : When an element is clicked.
- `ondblclick` : When an element is double-clicked.
- `onmouseover` : When the mouse pointer moves over an element.
- `onmouseout` : When the mouse pointer leaves an element.
- `onkeydown` : When a keyboard key is pressed.
- `onchange` : When an input value change is committed.

```
<button onclick="showMessage()">Click Me</button>

<p onmouseover="this.style.color='red'"
   onmouseout="this.style.color='black'">
    Move your mouse over this text!
</p>

<input type="text" onchange="alert('Value changed!')">

<script>
    function showMessage() {
        alert("Button clicked!");
    }
</script>
```

The addEventListener() Method

The `addEventListener()` method attaches an event handler to an HTML element without writing JavaScript directly in HTML attributes.

Syntax

```
element.addEventListener("event", function);
```

- **element**: The HTML element to monitor.
- **event**: The event type, such as "click".
- **function**: The function to execute when the event occurs.

Advantages of addEventListener()

- Separates HTML from JavaScript.
- Allows multiple listeners for the same event.
- Supports different event types.
- Provides additional control over event handling.

HTML Event Attribute	addEventListener()
<button onclick="greet()">	button.addEventListener("click", greet)
JavaScript written in HTML	JavaScript written separately
Limited to one handler per event attribute	Supports multiple listeners

```
<button id="myButton">Click Me</button>

<script>
    const button = document.getElementById("myButton");

    button.addEventListener("click", function() {
        alert("Button clicked!");
    });

// Using a Named Function
    function showMessage() {
        alert("Hello JavaScript!");
    }

    button.addEventListener("click", showMessage);
</script>
```

Notice that we write `showMessage,` not `showMessage()`, because we want to pass the function to the event listener rather than execute it immediately.

Amjad

Mouse and Keyboard Events

JavaScript can respond to both mouse and keyboard interactions.

Mouse Events

Event	Description
click	User clicks an element
dblclick	User double-clicks an element
mouseenter	Pointer enters an element
mouseleave	Pointer leaves an element
mousemove	Pointer moves within an element

Keyboard Events

Event	Description
keydown	A key is pressed
keyup	A key is released

```
// Example 1: Mouse Event
const box = document.getElementById("box");

box.addEventListener("mouseenter", function() {
    box.style.backgroundColor = "yellow";
});

box.addEventListener("mouseleave", function() {
    box.style.backgroundColor = "white";
});

// Example 2: Keyboard Event

document.addEventListener("keydown", function(event) {
    console.log("Key pressed: " + event.key);
});
```

The Event Object and Practical Examples

When an event occurs, JavaScript provides an event object containing information about that event.
Common properties and methods include:

Property / Method	Description
event.type	Type of event
event.target	Element that originally triggered the event
event.currentTarget	Element whose listener is currently executing
event.key	Keyboard key associated with a keyboard event
event.preventDefault()	Prevents the event's default browser action

```
<button id="counterButton">Click Me</button>
<p id="counter">0</p>

<script>
    let count = 0;

    const button = document.getElementById("counterButton");
    const counter = document.getElementById("counter");

    button.addEventListener("click", function() {
        count++;
        counter.textContent = count;
    });
</script>
```

What Happens?
1.The user clicks the button.
2.The browser detects the click event.
3.The event listener executes its function.
4.The counter increases by one.
5.The displayed number updates through DOM manipulation.

An-Najah National University



Chapter 8
Forms & Validation

Amjad



Introduction to JavaScript Form Handling

Form handling is the process of collecting, accessing, validating, and processing data entered by users into HTML forms.

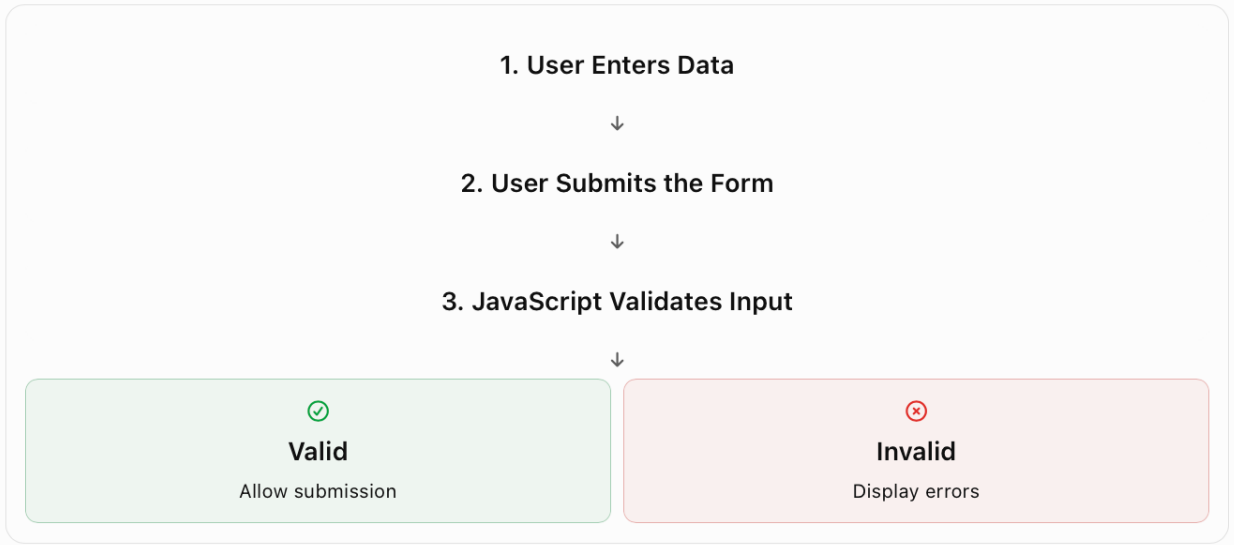
JavaScript allows us to:

- Read values entered into form fields.
- Respond to user interactions.
- Validate input before submission.
- Display error messages.
- Prevent invalid forms from being submitted.

```html
<form id="myForm">
    <label for="username">Username:</label>
    <input type="text" id="username">

    <label for="email">Email:</label>
    <input type="email" id="email">

    <button type="submit">Submit</button>
</form>
```



Reading Form Input Values

JavaScript uses the `.value` property to get the current value entered into an HTML input element.

```
<label for="name">Enter Your Name:</label>
<input type="text" id="name">

<button id="showButton">Show Name</button>

<p id="result"></p>

<script>
const nameInput = document.getElementById("name");
const button = document.getElementById("showButton");
const result = document.getElementById("result");

button.addEventListener("click", function() {
    const name = nameInput.value;
    result.textContent = "Hello, " + name;
});
</script>
```

Important Notes

- `.value` returns a string for text inputs.
- Use `.trim()` to remove leading and trailing whitespace.
- Use `.checked` to read the state of checkboxes.
- Use `Number()` to convert numeric input values when needed.

Form Submission and preventDefault()

What Happens When a Form Is Submitted?

- By default, submitting an HTML form causes the browser to perform its normal submission behavior, often navigating to another page or reloading the current one.
- JavaScript can intercept the submission using the submit event.

What Is preventDefault()?

The `event.preventDefault()` method prevents the browser from performing the event's default action.

```
<form id="loginForm">
    <input type="text" id="username" placeholder="Username">
    <button type="submit">Login</button>
</form>

<p id="message"></p>

<script>
const form = document.getElementById("loginForm");

form.addEventListener("submit", function(event) {
    event.preventDefault();

    const username = document.getElementById("username").value;

    document.getElementById("message").textContent =
        "Welcome, " + username;
});
</script>
```

Execution Steps

1.The user clicks the Submit button.
2.The browser triggers the submit event.
3.preventDefault() prevents the normal form submission.
4.JavaScript reads the entered username.
5.A welcome message appears without reloading the page.

JavaScript Form Validation

Form validation checks whether user input satisfies specific requirements before processing or submitting a form.

Common Validation Rules

Validation Type	Example
Required Field	Username cannot be empty
Minimum Length	Password must contain at least 8 characters
Email Format	Email must have a valid format
Numeric Range	Age must be between 18 and 60
Matching Values	Password and confirmation must match

Important Note

JavaScript validation improves user experience, but server-side validation is also necessary because client-side validation can be bypassed.

```
<input type="text" id="username">
<button id="checkButton">Validate</button>
<p id="message"></p>

<script>
const button = document.getElementById("checkButton");

button.addEventListener("click", function() {
    const username = document.getElementById("username").value.trim();
    const message = document.getElementById("message");

    if (username === "") {
        message.textContent = "Username is required!";
    } else if (username.length < 3) {
        message.textContent = "At least 3 characters required!";
    } else {
        message.textContent = "Valid username!";
    }
});
</script>
```

Displaying Validation Errors

How to Display Validation Messages

Instead of using alert(), JavaScript can display validation messages directly beside form fields.

This approach improves usability because users can immediately see which input needs correction.

```
<label for="email">Email:</label>
<input type="email" id="email">

<p id="error"></p>

<button id="validateButton">Validate</button>

<script>
const button = document.getElementById("validateButton");
const email = document.getElementById("email");
const error = document.getElementById("error");

button.addEventListener("click", function() {
    if (email.value.trim() === "") {
        error.textContent = "Email is required!";
        error.style.color = "red";
    } else if (!email.validity.valid) {
        error.textContent = "Enter a valid email address!";
        error.style.color = "red";
    } else {
        error.textContent = "Valid email!";
        error.style.color = "green";
    }
});
</script>
```

Complete Form Validation Example

Student Registration Form

```
<form id="registrationForm" novalidate>
    <label for="name">Full Name:</label>
    <input type="text" id="name">

    <label for="email">Email:</label>
    <input type="email" id="email">

    <label for="age">Age:</label>
    <input type="number" id="age">

    <button type="submit">Register</button>
</form>

<p id="message"></p>

<script>
const form = document.getElementById("registrationForm");
const message = document.getElementById("message");

form.addEventListener("submit", function(event) {
    event.preventDefault();

    const name = document.getElementById("name").value.trim();
    const emailInput = document.getElementById("email");
    const email = emailInput.value.trim();
    const ageInput = document.getElementById("age");
    const age = Number(ageInput.value);

    if (name === "" || email === "" || ageInput.value.trim() === "") {
        message.textContent = "All fields are required!";
        message.style.color = "red";
    } else if (!emailInput.validity.valid) {
        message.textContent = "Enter a valid email address!";
        message.style.color = "red";
    } else if (age < 18 || age > 60) {
        message.textContent = "Age must be between 18 and 60!";
        message.style.color = "red";
    } else {
        message.textContent = "Registration successful!";
        message.style.color = "green";
    }
});
</script>
```



Chapter 9
Asynchronous

Amjad



Synchronous vs. Asynchronous JavaScript

What Is Synchronous JavaScript?
In synchronous execution, JavaScript executes statements one after another. Each statement must finish before the next one can execute.

What Is Asynchronous JavaScript?
Asynchronous programming allows JavaScript to start certain operations and continue executing other code while waiting for those operations to complete.

Important: Asynchronous does not necessarily mean multiple JavaScript statements execute simultaneously. The browser coordinates operations and schedules callbacks using the event loop.

```
// Synchronous Example
console.log("Step 1");
console.log("Step 2");
console.log("Step 3");

/*
Step 1
Step 2
Step 3
*/

// Asynchronous Example
console.log("Step 1");

setTimeout(function() {
    console.log("Step 2");
}, 2000);

console.log("Step 3");

/*
Step 1
Step 3
Step 2
*/
```

Anjad

JavaScript Timers

JavaScript provides built-in timer functions to execute code after a delay or repeatedly.

1. setTimeout()

Executes a function once, after a specified delay.

```
console.log("Starting...");

setTimeout(function() {
    console.log("Hello after 3 seconds!");
}, 3000);

console.log("Continuing...");
```

Function	Purpose
setTimeout()	Schedule a function once
setInterval()	Schedule a function repeatedly
clearTimeout()	Cancel a scheduled timeout
clearInterval()	Stop a repeating interval

2. setInterval()

Executes a function repeatedly, with a specified delay between scheduled executions.

```
let count = 0;

const timer = setInterval(function() {
    count++;
    console.log(count);

    if (count === 5) {
        clearInterval(timer);
    }
}, 1000);
```

Output: The numbers 1 through 5 are printed, approximately one second apart.

Callback Functions

- A callback function is a function passed as an argument to another function, so that the receiving function can call it later or when appropriate.
- Callbacks are frequently used in asynchronous operations and event handling.

```
function greet(name, callback) {
    console.log("Hello, " + name);
    callback();
}

function sayGoodbye() {
    console.log("Goodbye!");
}

greet("Ali", sayGoodbye);

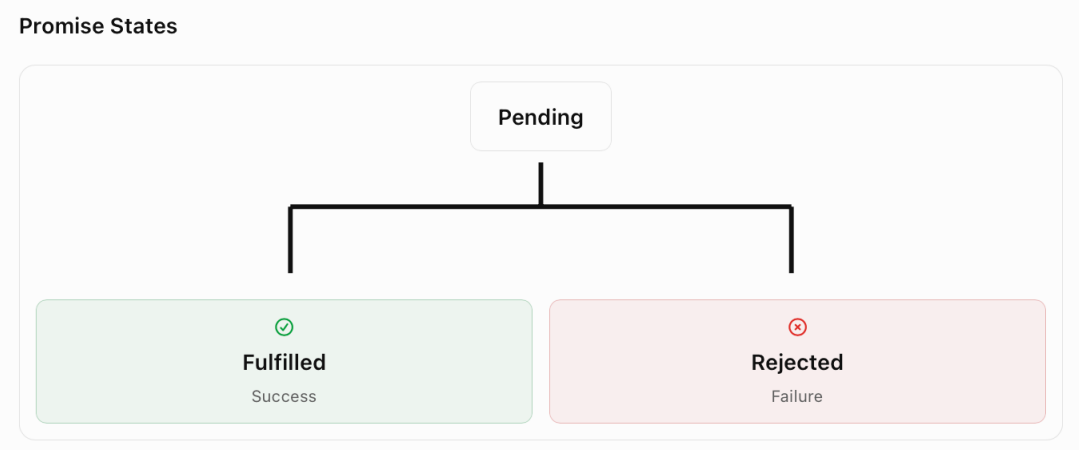
// Hello, Ali
// Goodbye!
```

JavaScript Promises

A Promise is an object representing the eventual completion or failure of an asynchronous operation.

A promise has three possible states:

- Pending: The operation has not completed.
- Fulfilled: The operation completed successfully.
- Rejected: The operation failed.



Important Methods

Method	Description
.then()	Handles a fulfilled promise
.catch()	Handles a rejected promise
.finally()	Executes when the promise settles, regardless of outcome

```
//Creating a Promise
const myPromise = new Promise(function(resolve, reject) {
    const success = true;

    if (success) {
        resolve("Operation successful!");
    } else {
        reject("Operation failed!");
    }
});

//Handling a Promise
myPromise
    .then(function(result) {
        console.log(result);
    })
    .catch(function(error) {
        console.log(error);
    });
```

Async and Await

async and await provide a cleaner way to work with promises.

- async declares an asynchronous function that always returns a Promise.
- await waits for a promise to settle within an async function, suspending that function's execution without blocking the entire JavaScript environment.
- try...catch can be used to handle errors.

Promise vs. Async/Await

Promises with .then()	Async/Await
Uses .then() to handle results	Uses await to obtain results
Uses .catch() for errors	Commonly uses try...catch
Uses callback-based chaining	Often reads more like sequential code

```
async function loadData() {
    const result = await Promise.resolve("Data loaded!");
    console.log(result);
}

loadData();

//Handling Errors
async function loadData() {
    try {
        const result = await Promise.reject("Network error!");
        console.log(result);
    } catch (error) {
        console.log(error);
    }
}

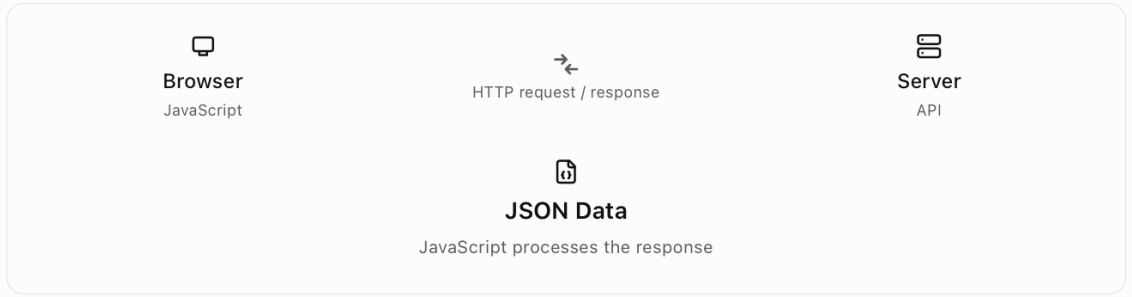
loadData();
```

Fetch API: Getting Data from a Server

The **Fetch API** allows JavaScript to make HTTP requests and retrieve resources from servers.

It is commonly used to:

- Retrieve data from a REST API.
- Send information to a server.
- Load data without reloading the web page.
- Communicate with backend applications.




```
async function getUsers() {
    try {
        const response = await fetch(
            "https://jsonplaceholder.typicode.com/users"
        );

        if (!response.ok) {
            throw new Error("HTTP error: " + response.status);
        }

        const users = await response.json();

        console.log(users);
    } catch (error) {
        console.log("Error:", error);
    }
}

getUsers();
```

Author

Amjad Ataallah

Email: amjad.atallah@najah.edu
Linkedin: https://www.linkedin.com/in/ataallah/
Website: https://amjadataallah.github.io

Yanal Sweseh

Email: yanal.swaseh@najah.edu
Linkedin: https://www.linkedin.com/in/yanalsweseh/
Website: https://github.com/yswesieh